

En el desarrollo de aplicaciones modernas con TypeScript y Node.js, la elección del ORM o *Query Builder* resulta determinante para el rendimiento, la mantenibilidad del código y la velocidad de ejecución. Tradicionalmente, los ORM del ecosistema imponían abstracciones complejas que ocultaban el funcionamiento interno de SQL, introduciendo problemas de rendimiento en arquitecturas *serverless* y generando una brecha entre las consultas de la aplicación y el motor de base de datos.

El objetivo de este documento es evaluar Drizzle ORM mediante una prueba de concepto (PoC). A lo largo del informe se analiza su propuesta de valor: un ORM ligero, rápido y con seguridad de tipos (*Type-Safe*) estricta que abraza la sintaxis de SQL puro. Se detallarán sus mecanismos de definición de esquemas, las estrategias de consulta disponibles (SQL-like y Relational Queries), la optimización del rendimiento mediante *Prepared Statements*, el flujo de migraciones, así como el manejo de transacciones y herramientas de administración como *Drizzle Studio*.

Definición de esquema y relaciones

Declaración de tablas

Una tabla representa una tabla de la base de datos PostgreSQL.

En Drizzle, esa tabla se define mediante código TypeScript utilizando `pgTable()`.

```
import { pgTable, serial, varchar } from "drizzle-orm/pg-core";

export const users = pgTable("users", {
  id: serial("id").primaryKey(),
  name: varchar("name", { length: 255 }).notNull(),
  email: varchar("email", { length: 255 }).notNull().unique(),
});
```

Tipado estricto

¿Qué es el tipado estricto?

El tipado estricto permite que TypeScript controle que los datos utilizados en el programa sean compatibles con los tipos definidos.

Esto permite detectar errores durante el desarrollo, antes de ejecutar el programa.

¿Qué es la inferencia de tipos?

La inferencia de tipos es la capacidad de TypeScript y Drizzle de determinar automáticamente los tipos de los datos a partir de la definición del esquema.

Por lo tanto, no es necesario definir manualmente una interfaz para cada tabla.

```
import { InferSelectModel, InferInsertModel } from "drizzle-orm";

type User = InferSelectModel<typeof users>;

type NewUser = InferInsertModel<typeof users>;

const nuevoUsuario: NewUser = {
  name: "Ana",
  email: "ana@mail.com",
};
```

Relaciones

¿Qué es una relación?

Una relación representa la asociación entre registros de diferentes tablas de una base de datos relacional.

Por ejemplo, una tabla puede estar relacionada con otra mediante una clave foránea (Foreign Key).

Foreign Key

¿Qué es?

Una Foreign Key es una columna que referencia la clave primaria de otra tabla.

Su función es establecer una relación entre ambas tablas y mantener la integridad referencial de los datos.

One-to-many

Un registro de una tabla puede estar relacionado con muchos registros de otra tabla, mientras que cada uno de esos registros pertenece a un único registro de la primera tabla.

```
import { pgTable, serial, varchar, integer, text } from "drizzle-orm/pg-core";
import { relations } from "drizzle-orm";

export const posts = pgTable("posts", {
  id: serial("id").primaryKey(),
  title: text("title").notNull(),
  authorId: integer("author_id").references(() => users.id),
});

export const usersRelations = relations(users, ({ many }) => ({
  posts: many(posts),
}));

export const postsRelations = relations(posts, ({ one }) => ({
  author: one(users, {
    fields: [posts.authorId],
    references: [users.id],
  }),
}));
```

Many-to-one

Un registro de una tabla puede estar relacionado con muchos registros de otra tabla y, al mismo tiempo, cada registro de la segunda tabla puede estar relacionado con muchos registros de la primera.

```
export const groups = pgTable("groups", {
  id: serial("id").primaryKey(),
  name: text("name").notNull(),
});

export const usersToGroups = pgTable("users_to_groups", {
  userId: integer("user_id").references(() => users.id),
  groupId: integer("group_id").references(() => groups.id),
});

export const usersToGroupsRelations = relations(usersToGroups, ({ one }) => ({
  user: one(users, { fields: [usersToGroups.userId], references: [users.id] }),
  group: one(groups, { fields: [usersToGroups.groupId], references: [groups.id] }),
})));
```

Tipos de consultas en Drizzle ORM

Drizzle ORM proporciona distintas formas de interactuar con la base de datos, diseñadas para combinar la flexibilidad del lenguaje SQL con las ventajas del tipado estático de TypeScript. Según la documentación oficial, las consultas pueden realizarse principalmente mediante la **SQL-like API** y la **Relational Queries API**.

SQL-like API

La **SQL-like API** constituye la forma principal de construir consultas en Drizzle. Su diseño busca mantener una sintaxis muy cercana a SQL, permitiendo al desarrollador expresar operaciones de selección, inserción, actualización y eliminación de forma intuitiva y con un alto nivel de control sobre la consulta generada.

A diferencia de otros ORM que abstraen completamente el lenguaje SQL, Drizzle conserva una estructura familiar para quienes ya poseen conocimientos de bases de datos relacionales. Al mismo tiempo, aprovecha el sistema de tipos de TypeScript para validar nombres de tablas, columnas y tipos de datos durante la compilación, reduciendo la posibilidad de errores antes de ejecutar la aplicación.

```
await db
  .select()
  .from(posts)
  .leftJoin(comments, eq(posts.id, comments.post_id))
  .where(eq(posts.id, 10))
```

La consulta SQL equivalente sería:

```
SELECT *  
FROM "posts"  
LEFT JOIN "comments" ON "posts"."id" = "comments"."post_id"  
WHERE "posts"."id" = 10
```

Este enfoque permite mantener una correspondencia clara entre el código de la aplicación y las sentencias SQL que finalmente ejecuta el motor de base de datos.

Relational Queries API

Además de las consultas tradicionales, Drizzle incorpora la **Relational Queries API**, una capa que aprovecha las relaciones definidas entre las entidades para simplificar la recuperación de información relacionada.

Cuando las relaciones entre tablas han sido declaradas en el esquema, el desarrollador puede solicitar los datos asociados sin escribir manualmente las cláusulas **JOIN**. Drizzle utiliza dicha información para construir automáticamente las consultas SQL necesarias.

Por ejemplo:

```
const result = await db.query.users.findMany({  
  with: {  
    posts: true  
  },  
});
```

Internamente, Drizzle genera una consulta equivalente a:

```
SELECT *  
FROM "posts"  
LEFT JOIN "comments" ON "posts"."id" = "comments"."post_id"  
WHERE "posts"."id" = 10
```

Es importante destacar que esta API no reemplaza el funcionamiento de SQL, sino que automatiza la construcción de consultas complejas a partir de las relaciones previamente definidas, mejorando la legibilidad del código y reduciendo la cantidad de instrucciones necesarias para acceder a información relacionada.

Prepared Statements

Un Prepared Statement consiste en preparar una consulta una única vez para que posteriormente pueda reutilizarse modificando únicamente los parámetros de entrada. De esta manera, la estructura de la consulta permanece constante mientras que únicamente cambian los datos utilizados en cada ejecución.

En Drizzle, un Prepared Statement puede definirse de la siguiente forma:

```
const prepared = db
  .select()
  .from(users)
  .where(eq(users.id, sql.placeholder("id")))
  .prepare();
```

Posteriormente, la consulta se ejecuta indicando únicamente el valor del parámetro:

```
await prepared.execute({ id: 5 });

await prepared.execute({ id: 10 });
```

Desde el punto de vista del motor de base de datos, el funcionamiento es equivalente a preparar previamente una sentencia SQL parametrizada:

```
1 SELECT *
2 FROM users
3 WHERE id = ?;
```

En cada ejecución únicamente cambia el valor asociado al parámetro, evitando que la consulta deba volver a analizarse y planificarse completamente.

Esta estrategia resulta especialmente beneficiosa en aplicaciones que ejecutan la misma consulta de forma repetitiva, como APIs REST, aplicaciones web con alto volumen de solicitudes o entornos *serverless*. Al reutilizar la consulta preparada, se reduce el tiempo dedicado al análisis sintáctico y a la generación del plan de ejecución, mejorando el rendimiento general de la aplicación.

MIGRACIONES

A diferencia de los ORM tradicionales que utilizan lenguajes de dominio específico (DSL), Drizzle define la estructura de datos directamente en TypeScript (`schema.ts`). Para trasladar estas definiciones a la base de datos relacional de manera segura y trazable, se utiliza el proceso de migraciones dividido en tres fases:

1. Declaración del esquema: Se definen las tablas y campos mediante código TypeScript fuertemente tipado.

```
import { pgTable, serial, text, integer } from 'drizzle-orm/pg-core';

export const users = pgTable('users', {
  id: serial('id').primaryKey(),
  firstName: text('first_name'),
  lastName: text('last_name'),
  address: text('address'),
  city: text('city'),
  age: integer('age'),
});
```

2. Generación del DDL (Drizzle kit generate): Al momento de ejecutar el comando “drizzle-kit generate” Drizzle Kit analiza la diferencia entre la versión anterior (si es que la hay) y el estado actual del archivo schema.ts. Como resultado, genera un archivo de migración en lenguaje SQL puro dentro de una carpeta local (ejemplo: ./drizzle/0000_create_users.sql). El propósito de todo esto es auditar las sentencias SQL antes de que alteren la base de datos, y así poder arrastrar algún error.

SQL

```
CREATE TABLE "users" (
  "id" SERIAL PRIMARY KEY,
  "first_name" TEXT,
  "last_name" TEXT,
  "address" TEXT,
  "city" TEXT,
  "age" INTEGER
);
```

3. Aplicación de cambios (drizzle-kit migrate): Se ejecutan los archivos SQL pendientes en el motor de base de datos de destino. Este proceso asegura la persistencia de los datos existentes sin pérdida ni alteración no deseada durante las modificaciones estructurales. En resumen, crea las tablas o las sentencias en tu base de datos.

Este formato es fácil de entender y revisar, porque lo pasa a sql directo sin intermediarios.

DRIZZLE STUDIO

Para complementar el flujo de trabajo, Drizzle incorpora **Drizzle Studio**, una herramienta gráfica que se ejecuta localmente desde la terminal (npk drizzle-kit studio).

Este permite examinar, filtrar y editar los registros de la base de datos mediante una interfaz gráfica tipo tabla. No permite crear tablas, solo interactuar con ellas.

The screenshot shows the Drizzle Studio application interface. On the left is a sidebar with a search bar and a list of database tables: Customers (93), Employees (9), Order Details (2155), Orders (830), Products (77), Shippers (3), and Suppliers (29). The 'Customers' table is selected. The main area displays a table of customer records with columns: CustomerID, CompanyName, ContactName, ContactTitle, Address, City, PostalCode, and Region. The table contains 50 records, with the first 10 visible. At the top right, there are controls for filters, columns, and a record count of 50.

CustomerID	CompanyName	ContactName	ContactTitle	Address	City	PostalCode	Region
ALFKI	Alfreds Futterkis...	Maria Anders	Sales Representative	Obere Str. 57	Berlin	12209	Western Euro
ANATR	Ana Trujillo Empa...	Ana Trujillo	Owner	Avda. de la ...	México D.F.	05021	Central Amer
ANTON	Antonio Moreno Ta...	Antonio Moreno	Owner	Mataderos 23...	México D.F.	05023	Central Amer
AROUT	Around the Horn	Thomas Hardy	Sales Representative	120 Hanover ...	London	WA1 1DP	British Isle
BERGS	Berglunds snabbköp	Christina Berglund	Order Administrator	Berguvsvägen...	Luleå	S-958 22	Northern Eur
BLAUS	Blauer See Delika...	Hanna Moos	Sales Representative	Forsterstr. ...	Mannheim	68306	Western Euro
BLONP	Blondesddsl père ...	Frédérique Citeaux	Marketing Manager	24, place Kl...	Strasbourg	67000	Western Euro
BOLID	Bólido Comidas pr...	Martín Sommer	Owner	C/ Araquil, ...	Madrid	28023	Southern Eur
BONAP	Bon app'	Laurence Lebihan	Owner	12, rue des ...	Marseille	13008	Western Euro
BOTTM	Bottom-Dollar Mar...	Elizabeth Lincoln	Accounting Manager	23 Tsawassen...	Tsawassen	T2F 8M4	North Americ
BSBEV	B's Beverages	Victoria Ashworth	Sales Representative	Fauntleroy C...	London	EC2 5NT	British Isle
CACTU	Cactus Comidas pa...	Patricio Simpson	Sales Agent	Cerrito 333	Buenos Aires	1010	South Americ
CENTC	Centro comercial ...	Francisco Chang	Marketing Manager	Sierras de G...	México D.F.	05022	Central Amer
CHOPS	Chop-suey Chinese	Yang Wang	Owner	Hauptstr. 29	Bern	3012	Western Euro
COMM1	Comércio Mineiro	Pedro Afonso	Sales Associate	Av. dos Lusí...	Sao Paulo	05432-043	South Americ
CONSH	Consolidated Hold...	Elizabeth Brown	Sales Representative	Berkeley Gar...	London	WX1 6LT	British Isle
DRACD	Drachenblut Delik...	Sven Ottlieb	Order Administrator	Walserweg 21	Aachen	52066	Western Euro
DUMON	Du monde entier	Janine Labrune	Owner	67, rue des ...	Nantes	44000	Western Euro

TRANSACCIONES

En cualquier aplicación real necesitás garantizar la consistencia de los datos. Por ejemplo, al procesar un pago o actualizar múltiples registros a la vez, no podés permitir que la primera parte se guarde si la segunda falla.

A nivel de consistencia de datos, Drizzle nos permite manejar transacciones complejas de forma declarativa con **db.transaction()**. Si cualquier operación falla dentro del bloque, Drizzle ejecuta un rollback automático garantizando que la base de datos nunca quede en un estado inconsistente o corrupto. Este tiene la ventaja que se conecta directo con la base de datos, sin nada de por medio, lo que lo hace mucho más rápido y no sufre cierres inesperados por tiempo.


```

await db.transaction(async (tx) => {
  // 1. Restar saldo al usuario
  await tx.update(users)
    .set({ balance: balance - 100 })
    .where(eq(users.id, 1));

  // 2. Crear la orden de compra
  await tx.insert(orders).values({
    userId: 1,
    total: 100,
  });
});

```

¿En qué casos conviene utilizar Drizzle?

A partir de las características analizadas, Drizzle resulta una alternativa especialmente interesante cuando:

- El equipo trabaja principalmente con TypeScript y Node.js.
- Los desarrolladores poseen conocimientos de SQL.
- Se busca mantener una relación cercana entre el código y las consultas SQL.
- Se necesita un sistema de tipos integrado con la definición del esquema.
- Se requiere controlar de forma explícita las consultas realizadas a la base de datos.
- Se utilizan consultas parametrizadas de manera frecuente.
- Se desea disponer de migraciones basadas en SQL que puedan ser revisadas y versionadas.

¿En qué casos puede no ser la mejor alternativa?

Drizzle puede no ser la opción más adecuada cuando:

- El equipo busca una abstracción elevada sobre SQL.
- El proyecto está fuertemente basado en entidades y su ciclo de vida.
- Se necesita una arquitectura centrada en patrones como Unit of Work e Identity Map.
- Los desarrolladores tienen poca experiencia trabajando con bases de datos relacionales y SQL.
- Las necesidades del proyecto se adaptan mejor a un ORM orientado a entidades.

En consecuencia, la elección de Drizzle no debería basarse únicamente en que sea una herramienta ligera o cercana a SQL. Su conveniencia depende de las necesidades del proyecto y de las prioridades del equipo de desarrollo.